TDD & Conway's Game of Life

We all need coding practice -- this is one way that we get better as programmers. Recently, I picked Conway's Game of Life (it's pretty cool when you Google it: there's an in-browser version of it). I've always been interested in this -- primarily because I find watching the patterns to be pretty mesmerizing. Check out the Wikipedia article to see some of these patterns.

I'm looking for some TDD practice, so we'll create a method that implements the rules. We won't hook up a UI to this (at least not yet), but we'll get the "business logic" working.

If you're not familiar with TDD (Test-Driven Development), we're going to follow a Red-Green-Refactor pattern. First we'll write a failing unit test (Red), then we'll write just enough code to get the test to pass (Green), and then we'll Refactor our code to make things easier to understand.

We'll use MSTest as our test platform. I like to start here because it's built in and the test runner integrates well with Visual Studio. But we'll see some limitations as we go. In a future article, we'll see that NUnit doesn't have these same limitations and lets us do some cool stuff (but wait for next time for that).

Completed code can be downloaded here: http://www.jeremybytes.com/Downloads.aspx#ConwayTDD
[Editor's Note: I fixed the bad link here - it was pointing to "localhost". D'oh]

[Note: If you'd rather watch a video, check out TDD: Don't Turn Off Your Brain.]

Conway's Game of Life

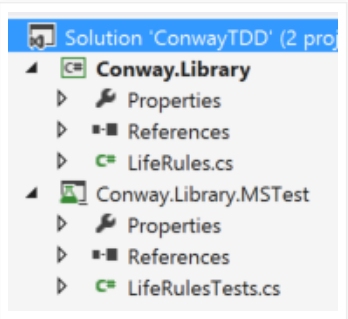
The rules for Conway's Game of Life are pretty simple:

1. Any live cell with fewer than two live neighbours dies, as if caused by under-population.
2. Any live cell with two or three live neighbours lives on to the next generation.
3. Any live cell with more than three live neighbours dies, as if by overcrowding.
4. Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.

We can create a method that takes in the current cell state (live/dead) and the number of live neighbors and then spits out a new state (live/dead). As a note, the rules have the British spelling "neighbours", but I'll be using the American spelling "neighbors" throughout the article.

Getting Started

To get started, I created a class library to hold our method and an MSTest project to hold our unit tests. I know that technically I'm supposed to write a failing test before writing *any* code, but I like to break rules from time to time. So, I created a starting class:



This shows us the "LifeRules.cs" file that will hold our code (again, this is just an empty class right now), and the "LifeRulesTests.cs" file that will hold our unit tests.

Let's implement our rules one by one, and we'll see how things go. For this, I'll just copy the rules into the test file:

```
[TestClass]
public class LifeRulesTests
{
    //Any live cell with fewer than two live neighbours dies.
    //Any live cell with two or three live neighbours lives.
    //Any live cell with more than three live neighbours dies.
    //Any dead cell with exactly three live neighbours becomes a live cell.

}
```

Now let's write some tests.

Test #1

For our first test, we need to test that a live cell with fewer than two live neighbors dies. Now, I'm going to "cheat" a bit more in my TDD process. I'm going to stub out the method and parameters before writing my first test. This doesn't bother me in this case. I like the idea of building tests first, but there's a bit of practical stubbing that makes things easier.

```
namespace Conway.Library
{
    public enum CellState
    {
        Alive,
        Dead,
    }

    public class LifeRules
    {
        public static CellState GetNewState(CellState currentState, int liveNeighbors)
        {
            return currentState;
        }
    }
}
```

Here's the decisions I made. First, I created a CellState enum to handle the "Alive" or "Dead" state for the current cell. Then I created a method that takes the state of the current cell along with the number of live neighbors and returns a new state.

As a starting point, we just return the current state of the cell unchanged.

Now that we have a little bit of a framework, it's easy to write a test for our first rule:

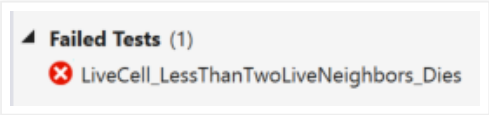
```
[TestMethod]
public void LiveCell_LessThanTwoLiveNeighbors_Dies()
{
    //Any live cell with fewer than two live neighbors dies.
    //Arrange
    CellState currentState = CellState.Alive;
    int liveNeighbors = 1;

    // Act
    CellState result = LifeRules.GetNewState(currentState, liveNeighbors);

    // Assert
    Assert.AreEqual(CellState.Dead, result);
}
```

I like to name my tests based on a 3-part scheme recommended by Roy Osherove (who wrote *The Art of Unit Testing*). The first part is the unit under test -- in this case we're testing a living cell. The second part is the action we're performing -- calling our method with less than 2 living neighbors. The third part is the expected result -- the cell dies. This matches the first rule that we're testing.

[Grammatical Note: The English major in me wants to say "fewer than" since we're dealing with countable items, but the programmer in me says "less than" since that's the name of our comparison operator.]

To arrange this, we set up the current state as "Alive" and say that we have "1" live neighbor. For our action, we call "GetNewState" with these parameters. And then we check our result. In this case, we're expecting that the result will be "Dead".

This test fails:



Before going any further, you're probably saying that my test doesn't really match the description. Although the test says "less than 2", our actual test case is 1. We'll loop back around to this in a bit. For now, let's get our test to pass.

Updating the Method
We're supposed to put in the bare minimum of code to get things to pass. This means that we should probably put in something like this (which tests the "1" case):

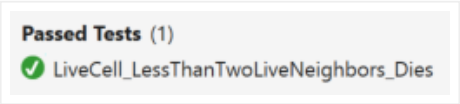
```
public static CellState GetNewState(CellState currentState, int liveNeighbors)
{
    if (currentState == CellState.Alive && liveNeighbors == 1)
        return CellState.Dead;

    return currentState;
}
```

But I'm going to put in the entire rule:

```
public static CellState GetNewState(CellState currentState, int liveNeighbors)
{
    if (currentState == CellState.Alive && liveNeighbors < 2)
        return CellState.Dead;

    return currentState;
}
```

When we're doing TDD, it's up to us to decide how big or small our chunks are. I'm comfortable biting off fairly large chunks to start off with. If I find myself getting into trouble, then I'll break things down a bit smaller.

With this in place, our test passes:



Success!

Test #2
Let's move on to the next rule: any live cell with 2 or 3 neighbors stays alive.

```
[TestMethod]
public void LiveCell_TwoOrThreeLiveNeighbors_Lives()
{
    // Any live cell with two or three live neighbors lives.
    // Arrange
    CellState currentState = CellState.Alive;
    int liveNeighbors = 3;

    // Act
    CellState result = LifeRules.GetNewState(currentState, liveNeighbors);

    // Assert
    Assert.AreEqual(CellState.Alive, result);
}
```

Just like our other test, we set up the initial state, run the method, and check the results.

Let's run the tests:

Passed Tests (2)
- LiveCell_LessThanTwoLiveNeighbors_Dies
- LiveCell_TwoOrThreeLiveNeighbors_Lives

Well this is awkward. When we're doing TDD, we expect Red-Green-Refactor. This test went green immediately.

And that's okay. In our method, if we don't hit the conditional, it simply mirrors the incoming state. So our test immediately passes. We'll accept that and move on.

Test #3

For the third rule, we expect that a living cell with more than 3 neighbors will die from overcrowding.

```
[TestMethod]
public void LiveCell_MoreThanThreeNeighbors_Dies()
{
    // Any live cell with more than three live neighbors dies.
    // Arrange
    CellState currentState = CellState.Alive;
    int liveNeighbors = 4;

    // Act
    CellState result = LifeRules.GetNewState(currentState, liveNeighbors);

    // Assert
    Assert.AreEqual(CellState.Dead, result);
}
```

Again, our test looks pretty similar to the others. We're just changing our input state (4 live neighbors) and the expected result (Dead).

And this test fails:

Failed Tests (1)
- LiveCell_MoreThanThreeNeighbors_Dies

That's good. Let's adjust our code so that this passes.

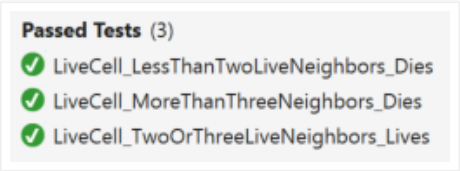
Updated Code

We'll just make a few tweaks here.

```
public static CellState GetNewState(CellState currentState, int liveNeighbors)
{
    if (currentState == CellState.Alive && liveNeighbors < 2)
        return CellState.Dead;
    if (currentState == CellState.Alive && liveNeighbors > 3)
        return CellState.Dead;

    return currentState;
}
```

We've added another conditional to check if the cell is currently alive and has more than 3 live neighbors. If this is the case, then the cell dies.

And our test now passes:



Let's move on to the last rule.

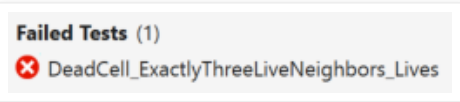
Test #4

I'm sure you're getting the idea now. The last rule states that if a cell is dead and there are exactly 3 neighbors, it comes back to life. The test is pretty easy to write:

```
[TestMethod]
public void DeadCell_ExactlyThreeLiveNeighbors_Lives()
{
    // Any dead cell with exactly three live neighbors becomes a live cell.
    // Arrange
    CellState currentState = CellState.Dead;
    int liveNeighbors = 3;

    // Act
    CellState result = LifeRules.GetNewState(currentState, liveNeighbors);

    // Assert
    Assert.AreEqual(CellState.Alive, result);
}
```

Not much different than the other tests -- just our initial state and expected results.

And the test fails:



Updated Code

Let's fix our method so that it can handle this new condition.

```
public static CellState GetNewState(CellState currentState, int liveNeighbors)
{
    if (currentState == CellState.Alive && liveNeighbors < 2)
        return CellState.Dead;
    if (currentState == CellState.Alive && liveNeighbors > 3)
        return CellState.Dead;
    if (currentState == CellState.Dead && liveNeighbors == 3)
        return CellState.Alive;

    return currentState;
}
```

We just added another conditional. And now our test passes.

Passed Tests (4)
- DeadCell_ExactlyThreeLiveNeighbors_Lives
- LiveCell_LessThanTwoLiveNeighbors_Dies
- LiveCell_MoreThanThreeNeighbors_Dies
- LiveCell_TwoOrThreeLiveNeighbors_Lives

Refactoring

The parts of TDD are Red-Green-Refactor. So far we've seen plenty of Red (initial failing tests) and Green (passing tests after updating code), but we haven't done any Refactoring yet.

I'm going to change our conditionals around a bit. The first two conditionals deal with changing "Alive" cells to "Dead" cells. The third conditional deals with a "Dead" cell becoming "Alive". Everything else just drops through, meaning the state remains unchanged.

Let's add a switch statement and combine some of the conditions:

```
public static CellState GetNewState(CellState currentState, int liveNeighbors)
{
    switch (currentState)
    {
        case CellState.Alive:
            if (liveNeighbors < 2 || liveNeighbors > 3)
                return CellState.Dead;
            break;
        case CellState.Dead:
            if (liveNeighbors == 3)
                return CellState.Alive;
            break;
    }

    return currentState;
}
```

I think this makes things a bit easier to follow. Not everyone may agree with that, but I like the switch with the conditionals better than the 3 separate conditionals.

We need to re-run our tests to make sure our refactoring didn't break anything:

Passed Tests (4)
- DeadCell_ExactlyThreeLiveNeighbors_Lives
- LiveCell_LessThanTwoLiveNeighbors_Dies
- LiveCell_MoreThanThreeNeighbors_Dies
- LiveCell_TwoOrThreeLiveNeighbors_Lives

And everything still passes.

Incomplete Tests

We've implemented all 4 of the rules, but we're not really testing all of the possible states at this point. For Test #1 (live cell with less than 2 neighbors dies), we're only testing for 1 neighbor. We really should add another test for 0 neighbors to make sure we're covering the possibilities.

In addition for Test #2 (live cell with 2 or 3 neighbors lives), we're only testing for 3 neighbors. We should add another test for 2.

The same is true for Test #3 (live cell with more than 3 neighbors dies). We're testing for 4 neighbors, but what about the others. The way the Game of Life is set up, a cell has 8 neighbors, so we should really test for 5, 6, 7, & 8 live neighbors, too.

And for Test #4 (dead cell with 3 neighbors comes to life), we should really test the negative conditions as well. All other states (less than 3 live neighbors or more than 3 live neighbors) and the dead cell stays dead.

That's a lot of tests.

Limitation with MSTest

Many testing frameworks have the concept of ~~data-driven or~~ *[see update]* parameterized tests -- that is, tests where we can vary the inputs without writing new tests. Unfortunately, MSTest is a bit incomplete in this regard. MSTest for Windows Store Apps does have this capability, but MSTest for Web/Desktop applications does not.

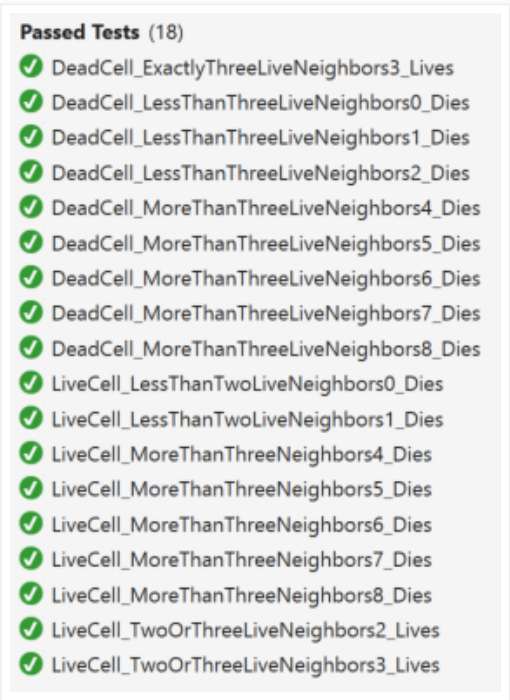
This is where I have to say "You've got to be kidding me, Microsoft." They obviously saw the value of parameterized tests, but not for all environments. (Now I do say this with a lot of love. Visual Studio is an awesome development environment. It's easy to complain about a lot of Microsoft products, but Visual Studio isn't one of them.)

NUnit is another testing tool (which is downloadable with NuGet) that does offer parameterized tests. And we'll look at this in the next article. For now, we'll just implement the tests manually.

*[**Update 10/28:** As Chris points out in the comments, MSTest does offer data-driven tests using the [DataSource] attribute. This allows you to hook up XML, CSV, SQL, or any other data source to hold test data. Here's an MSDN article, but you can find some easier tutorials online as well. With my memory jogged, I remember looking into this a few years back and not liking it because of the weird syntax to access the data inside the test methods themselves. So, I must have blocked it out. To make things interesting, data-driven tests are *not* available for Windows Store Apps (they use parameterized tests as mentioned above). I'd still like to see parameterized tests come in to all versions of MSTest so we can use the same methods across the board.]*

Completing the Test Scenarios

Since we're using MSTest here, we'll have to create separate test methods for each of these cases. I won't show you all of the code, but I will show you the Test Explorer with all of these tests.



The good news is that these additional test cases pass with our existing code. For the tests themselves, I renamed the things a bit. For the middle part of the method name, I added a number. This specifies the actual value that is being used for testing.

This isn't the greatest way of doing this. It's really a brute force method. But it does get the job done, and it covers all 18 test cases that we need.

Wrap Up

So we reached our goal: creating a method to process the rules of Conway's Game of Life. And we used TDD to get there. But our current set of tests isn't the most efficient way of testing things.

Since we do have so many scenarios that have the same expected outcome, it's really a shame to have separate test methods. It would be better to parameterize the tests in some way. We could probably do this ourselves (if we're really creative), but it would be better to use a testing framework that supports this. Next time, we'll take a look at NUnit. This allows us to test the same 18 cases with only 6 actual tests (technically, we could have fewer tests, but we'll talk about that, too). So, come back next time to see that in action.